[P3] Sixteen cards

Mohamed A. Osman — ORCID 0009-0004-5912-999X

Licence: CC BY 4.0. Research draft.

Abstract

No pairwise intersecting 6-uniform family of sixteen cards has transversal number six. A hypothetical counterexample is reduced to one of 463 canonical eight-card cores; for each core the completion problem is written as a CNF formula; all 463 are unsatisfiable; and each unsatisfiability carries a DRAT certificate accepted by an independent checker. Each computational step was also repeated by separately written code (§7). With [P2] this gives $g(6) \geq 17$. The fifteen-card exclusion also has a hand proof in [P1]; [P2] is still needed here for its Corollary 2, which fixes the maximum degree at four.

This is the computer-assisted stage of the project. The reduction and the encoding are proof obligations that live in the source, not in the certificates.

Computer-assisted; not independently reviewed. The reduction and the encoding are open obligations no certificate discharges. No priority claimed. See the note at the end.

1. Setting

Let F be a pairwise intersecting 6-uniform family of sixteen cards with no five-cover.

By [P2, Cor 2] the maximum symbol degree is exactly four.

Fix a symbol x of degree four and delete its four cards. Twelve cards remain; call them the **kernel** G.

Proposition 1. $\tau(G) = 5$.

Proof. If $\tau(G) \leq 4$ then x together with a four-cover of G is a five-cover of F. And deleting x from any of the four removed cards leaves five symbols meeting every card of G, since each card of G meets that card and cannot do so through x. □

So the four deleted cards are $\{x\} \cup C_i$ for $i = 1, \ldots, 4$, where each C_i is a five-element transversal of G. Every symbol of C_i occurs in G: a symbol outside G meets no card of G, and the other four symbols of C_i would then cover G.

Proposition 2 (extension criterion). *Four five-covers $C_1, \ldots, C_4$ of G produce a sixteen-card family with $\tau = 6$ if and only if every five-cover T of G is disjoint from at least one C_i.*

Proof. If some T met all four, then T would cover the twelve kernel cards and also the four new cards, a five-cover of F. Conversely, a five-cover of F containing x would cover G with at most four other symbols, contradicting Proposition 1; and a five-cover not containing x is a five-cover T of G, which by hypothesis misses one new card. Any single card is a six-cover, so $\tau(F) = 6$ exactly. □

In the language of graphs: form the **disjointness graph** on the five-covers of G, joining two when they are disjoint. Then $C_1, \ldots, C_4$ must be a *total dominating set* — every vertex, including a

selected one, needs a selected neighbour. This is not ordinary domination, where a selected vertex may dominate itself.

Degree capacities. For a symbol z of G, let $h(z)$ count its occurrences among the four selected covers. Since the final degree cannot exceed four,

$$d_G(z) + h(z) \le 4.$$

So a degree-four symbol of G cannot be used in any new card, a degree-three symbol occurs in at most one C_i, and a degree-two symbol in at most two. In particular every symbol of $C_i \cap C_j$ has kernel degree at most two.

2. The second deletion

The kernel G either contains a symbol of degree four or does not, and the two cases need different treatment. This section handles the first; §2a handles the second. Nothing below is reached when G has no degree-four symbol, so §2a is a necessary branch of the reduction and not a check on it.

Suppose y is a degree-four symbol of G. Its four cards are disjoint from the four cards of x, and deleting both leaves eight cards H.

Proposition 3. $\tau(H) = 4$ and the maximum degree in H is at most three.

Proof. $\tau(H) \ge 4$: a three-cover of H together with x and y would be a five-cover of F. $\tau(H) \le 4$ is [P2, Thm 1] applied after padding, or directly. If some symbol had degree four in H, deleting its cards would leave at most four, coverable by two symbols, so that symbol plus two plus x plus y is a five-cover. □

Proposition 4. *The degree-three symbols of H, read as triples of cards, are pairwise intersecting.*

Proof. Two disjoint triples cover six cards with two symbols; the remaining two cards share a symbol; together with x and y that is a five-cover. □

An eight-card core with these properties exists — see [P2, §5] for an explicit one — so this stage cannot be closed by a lemma the way eleven cards were. The cores must be enumerated.

2a. The second branch: kernels of maximum degree three

Now suppose G has no degree-four symbol, so every symbol of G has degree at most three. There is no y, no eight-card core, and none of §3 applies.

The branch is not empty. The twelve-card kernel of the seventeen-card witness of [P4] has six degree-two symbols and twenty degree-three symbols, so its maximum degree is three. It is one of the kernels enumerated below, and the enumeration finds it there.

The three profiles. Write (t, b, s) for the numbers of symbols of degree three, two and one in a card of G. Then $t + b + s = 6$, and the card must meet the other eleven, so $2t + b \ge 11$. The only solutions are $(5, 1, 0)$ and $(6, 0, 0)$; degree-one symbols are absent. Let c count the cards of the first type. Each of them holds exactly one degree-two symbol and each degree-two symbol lies in two cards, so c is even and there are $c/2$ degree-two symbols; the degree-three incidences number $72 - c$ and must be divisible by three. Hence c is a multiple of six, so $c \in \{0, 6, 12\}$ and the profiles are

c	degree-two symbols	degree-three symbols
12	6	20
6	3	22
0	0	24

One pair multigraph per profile. A $(6,0,0)$ card meets eleven others through six degree-three symbols reaching twelve, so it carries exactly one unit of intersection excess; a $(5,1,0)$ card reaches exactly eleven and carries none. So the excess graph E is a perfect matching on the $12-c$ cards of type $(6,0,0)$, and the degree-two graph F is a perfect matching on the c cards of type $(5,1,0)$; together they form one perfect matching on all twelve cards. The degree-three symbols therefore decompose the pair multigraph

$$M = K_{12} + E - F$$

into triangles, where the pairs of F are already served and those of E are served twice. Up to relabelling M depends only on c, so there are exactly three models, and a kernel of this branch is a triangle decomposition of one of them.

Enumeration. Every triangle decomposition of each M is enumerated, those with $\tau(G) \le 4$ are discarded — four symbols reach all twelve cards only as four disjoint triples that partition them, since $3+3+3+2 = 11 < 12$ leaves no room for a degree-two symbol — and the rest are reduced under $\mathrm{Aut}(M)$, which permutes the F-pairs among themselves and the E-pairs among themselves and may swap the two cards inside any pair.

profile	$\lvert\mathrm{Aut}(M)\rvert$	distinct decompositions	with $\tau(G) = 5$	kernels up to isomorphism	five-covers per kernel
(6, 20)	46,080	115,200	69,120	2	296
(3, 22)	2,304	1,498,752	558,672	259	363 – 381
(0, 24)	46,080	17,291,520	3,239,040	80	450 – 490

So there are **341** kernels of maximum degree three, up to isomorphism.

The test. On each of the 341, the extension criterion of Proposition 2 is solved exactly, subject to the degree caps $d_G(z) + h(z) \le 4$ of §1: four five-covers $C_1, \dots, C_4$ are sought whose disjointness-neighbourhoods together contain every five-cover of G. This is a set cover of the five-covers of G by four neighbourhoods, and it is searched to exhaustion.

Lemma 2 (computer-assisted). *No twelve-card kernel of maximum degree three admits four five-covers satisfying the extension criterion. Hence no sixteen-card counterexample has a kernel of this kind.*

Zero of the 341 admits such a quadruple. Script: `code/verify_degree3_kernels.py`, with the kernels in `data/degree3_kernels_c12.json`, `_c6.json` and `_c0.json` and an optional C accelerator in `code/fast_enumerate.c`.

The witness kernel of [P4] is among the two kernels of profile (6, 20), and it admits no four-cover extension — which is consistent with it extending to seventeen cards by five covers, as [P4, §4] describes, and not to sixteen by four.

With §2 and §2a the two branches are exhaustive: every sixteen-card counterexample either has a second degree-four symbol in its kernel, and then reduces to one of the 463 cores of §3, or does not, and is excluded by Lemma 2.

3. The 463 canonical cores

The enumeration runs in four steps, each with its own script and output.

Step 1. Maximal intersecting triple systems on eight points. Enumerate the maximal cliques of the graph whose vertices are the 56 triples from eight points and whose edges join intersecting triples, keeping those whose union is all eight points. There are **10,144**, of sizes {21:8, 16:336, 14:1680, 12:8120}. Script: `code/enumerate_eight_maximal.py`.

Step 2. Orbits. Under the symmetric group on the eight points these fall into **seven** orbits, of sizes

$$8, \quad 280, \quad 1680, \quad 56, \quad 560, \quad 2520, \quad 5040,$$

summing to 10,144. Script: `code/verify_eight_orbits.py`, which rebuilds each orbit from a representative and checks it equals the observed class.

Step 3. Triple systems. Every card of H holds a degree-three symbol: it must meet the other seven cards, and six symbols of degree at most two reach at most six of them. So the degree-three symbols cover all eight cards, and by Proposition 4 they lie inside one of the maximal families of Step 1. A triple can occur at most three times: a card holding m copies of one triple reaches the other

five cards through $6 - m$ symbols of degree at most three, so $2(6 - m) \geq 5$. For each of the seven representatives, enumerate the multisets of its triples, each of multiplicity at most three, that cover every card and leave every card room for the degree-two symbols its uncovered pairs need. That gives **39,768** triple systems in total, distributed over the seven representatives as 28329, 4932, 1526, 3858, 156, 535, 432. Reducing by the stabiliser leaves **129** symmetry classes, whose multiplicities sum back to 39,768. Scripts: `code/enumerate_eight_triples.py`, `code/classify_eight_triples.py`.

Step 4. Completions. Each class is completed with the degree-two symbols forced by the uncovered pairs, then with the remaining slack filled by binary and singleton symbols. That gives **772** raw completions, which reduce to **463** canonical cores. Script: `code/enumerate_eight_cores.py`; output `data/eight_cores.jsonl`.

Each line of that file records `supports`, a list of symbol supports as subsets of $\{0, \ldots, 7\}$, and a `multiplicity`. `code/verify_eight_orbits.py` also checks that every one of the 463 cores really is eight distinct six-element cards, pairwise intersecting, with no three symbols covering all eight.

Lemma 1 (finite reduction, computer-assisted). *Every hypothetical sixteen-card counterexample is represented by one of the 463 cores in `data/eight_cores.jsonl`.*

The completeness of this reduction is a proof obligation on the four scripts above. It is not certified by anything downstream.

4. The CNF encoding

For a fixed core, the remaining unknown is how the eight new cards — the four containing x and the four containing y — are filled.

Positions $0, \ldots, 7$ index the eight new cards, the first four being the x-block and the last four the y-block. The variables are:

- $h_{v,j}$ for each core symbol v and each new card j: the core symbol v also appears in new card j;
- z_b for each subset $b \subseteq \{0, \ldots, 7\}$ of size 2, 3 or 4 meeting both blocks: a fresh symbol whose support is exactly b.

The clauses enforce:

1. **degree caps** — core symbol v with existing support s may take at most $4 - |s|$ new cards, and a singleton must take at least one of them, since a symbol of degree one is impossible ([P1, §2]);
2. **card size** — for each new card j, exactly five of the core symbols and fresh symbols lie in it, the sixth being x or y;
3. **intersection with the core** — each new card meets every core card;
4. **cross-block intersection** — each x-card and each y-card share a core symbol or a fresh crossing symbol;
5. **antichain constraints** from Lemma 3 below: no fresh symbol's support contains another's, no fresh symbol's support lies inside a core symbol's completed support, and no core symbol's completed support lies inside another's (equal supports included);
6. **no five-cover** — for every four-element and every five-element subset of the symbol set that already covers the eight core cards, clauses forbidding it from also meeting all eight new cards;
7. **symmetry breaking** — lexicographic constraints along the chain $(0,1), (1,2), (2,3), (4,5), (5,6), (6,7), (0,4)$, which are legitimate because the four x-cards are interchangeable, the four y-cards are interchangeable, and x and y may be swapped.

Implementation: `code/search_sixteen.py`, function `build`. The same function exports the DIMACS files through `code/export_core_cnf.py`, so the CNF a reviewer checks is the one the search used.

Lemma 3 (support containment). *In a pairwise intersecting 6-uniform family with $\tau = 6$, no symbol's support is contained in another symbol's support.*

Proof. Suppose every card containing a also contains $b \neq a$, and let A be a card containing a. Every other card D meets A. If D meets A only in a, then D contains a, hence b, and $b \in A$; so D meets $A \setminus \{a\}$ in any case. Then the five symbols of $A \setminus \{a\}$ cover the family. □

Lemma 3 is what the clause families 1 and 5 rest on, and it also fixes the fresh symbols. A fresh symbol inside one block would have its support inside the support of x or of y; a fresh symbol of degree one is impossible; degree five is above the cap; and two fresh symbols with the same support are excluded. So the fresh symbols are exactly one z_b for each subset b of size 2, 3 or 4 meeting both blocks. For the no-five-cover clauses: x, y and the fresh symbols meet no core card, and no three core symbols cover the core, so a five-cover of the sixteen cards contains at least four core symbols that already cover the core. Clause family 6 lists exactly those cases.

The encoding is part of the mathematics. A DRAT certificate says the formula has no model; it says nothing about whether the formula means what §1 to §3 mean. That reading has to be done in the source.

5. Results

All 463 instances are unsatisfiable.

The first pass used a 50,000-conflict budget and returned 460 UNSAT and three UNKNOWN — cores 7, 239 and 342. Budget exhaustion is never treated as exclusion: those three were rerun without a budget and proved UNSAT. `code/resolve_sixteen.py` carries that rule explicitly.

core	variables	clauses	conflicts	solve time (s)
7	40,583	210,333	52,765	7.406
239	38,280	199,136	69,317	9.594
342	36,922	190,406	83,403	11.453

Raw records are in `results/sat_results_cadical195_0.jsonl` and the retry files.

6. DRAT certification

For each core the CNF was written in DIMACS form, a standalone CaDiCaL produced a DRAT proof, and `drat-trim` checked that proof independently. The batch finished with

```
VERIFIED: 462
SKIPPED : 1
FAILED  : 0
```

The skipped instance was core 0, which had already been verified separately before the batch, so the combined total is 463 of 463 verified with none failing. Per-core records are in `results/certificate_summary.jsonl`; the three heaviest are

core	DRAT bytes	checker	time (s)
7	10,110,384	VERIFIED	6.658
239	5,858,286	VERIFIED	2.792
342	7,661,933	VERIFIED	7.252

The CNF and DRAT files for those three are shipped in `data/cnf/` so that the certification step can be reproduced without a solver. The full archive of 463 pairs is large and belongs in a release asset or an archive record rather than in the repository.

7. The trusted boundary

A checked DRAT proof establishes that the generated formula is unsatisfiable. It removes the SAT solver from the trusted base. It does **not** establish any of:

- that every sixteen-card counterexample reaches one of the 463 cores;
- that `data/eight_cores.jsonl` is complete and free of accidental omission;
- that each CNF variable and clause has the intended combinatorial meaning.

Those three are the reduction and encoding obligations. They are audited through the source, the canonical data, the structural counts of §3, and the controls recorded in `AUDIT.md`. With them accepted, the 463 checked contradictions leave no counterexample.

A second route for each step. Each computational step has also been repeated by separately written code that shares nothing with the first route except the data it is asked to check:

- the cores were rebuilt straight from the definition of Step 3 — every labelled multiset of triples, with no maximal families and no orbits — and canonicalised under all 40,320 relabellings. This gives the same 129 classes, the same 772 completions, and the same 463 cores, with none extra on either side (`code/verify_cores_direct.py`);
- the 463 completion problems were encoded again from scratch, with a different cardinality encoding, the clauses of Lemma 3 written directly, and **no symmetry breaking at all**, and solved with a different solver. All 463 are UNSAT. With the no-five-cover clauses removed, the two encodings are satisfiable on exactly the same sample cores (`code/verify_sixteen_second_encoding.py`);
- the extension test of Lemma 2 was decided again by SAT, on a relaxation (at most four covers, repeats allowed). None of the 341 kernels extends; with five covers allowed, the [P4] kernel does (`code/verify_degree3_kernels_sat.py`).

This does not replace a reading of the reduction. The second route repeats the computations, not the arguments of §1 to §3, and the second solve carries no DRAT certificate. The enumeration of the 341 kernels themselves has not been repeated.

Theorem 1 (computer-assisted). *There is no pairwise intersecting 6-uniform family of sixteen cards with transversal number six. With [P2, Cor 1],* $g(6) \geq 17$.

The two branches of §2 and §2a together exhaust the possibilities: Lemma 2 closes the kernels of maximum degree three, and the 463 certified contradictions close the rest.

8. Verification

claim	script or data
the three profiles, 341 kernels of maximum degree three, no extension	`code/verify_degree3_kernels.py`
the same enumeration in C	`code/fast_enumerate.c`
10,144 maximal systems, sizes	`code/enumerate_eight_maximal.py`
seven orbits, sizes as listed	`code/verify_eight_orbits.py`
39,768 triple systems, 129 classes	`code/classify_eight_triples.py`, `data/eight_triple_classes.jsonl`
772 raw, 463 canonical cores	`code/enumerate_eight_cores.py`
every core is a genuine eight-card family with $\tau = 4$	`code/verify_eight_orbits.py`
the CNF is the one `build` produces	`code/verify_cnf_regeneration.py`
the encoding is not trivially over-constrained	`code/verify_encoding_controls.py`
463/463 UNSAT	`results/sat_results_cadical195_0.jsonl`, `results/sat_retry*.jsonl`
463/463 DRAT-verified	`results/certificate_summary.jsonl`, `data/cnf/`
second route: the 129 classes and the 463 cores from the definition	`code/verify_cores_direct.py`, `code/enumerate_triples_direct.c`
second route: 463/463 UNSAT, other encoding, no symmetry breaking	`code/verify_sixteen_second_encoding.py`, `results/second_encoding_cadical153.txt`
second route: no kernel of maximum degree three extends	`code/verify_degree3_kernels_sat.py`

This work was prepared with AI assistance (ChatGPT, OpenAI; Claude, Anthropic), used for algebraic derivation, for drafting and rewriting code and text, for running and re-running the computations, and for auditing the papers against their own scripts. The same tools were used in the review, so the review carries the same caveat. All statements were checked by the author, who is responsible for them. No priority is claimed and the result has not been independently reviewed by a human or a proof assistant; the repository README sets out the division of labour and the open obligations in full.

References

1. J. Barát and I. M. Wanless, *Intersecting and 2-intersecting hypergraphs with maximal covering number: the Erdős–Lovász theme revisited*, J. Combin. Des. 29 (2021), 260–286; arXiv:2011.04444.
2. V. Sivashankar, *An Improved Lower Bound for the Erdős–Lovász Cover Number Problem*, arXiv:2606.24878v2.
3. M. J. H. Heule, W. A. Hunt Jr., N. Wetzler, *Trimming while checking clausal proofs*, FMCAD 2013. The `drat-trim` checker.